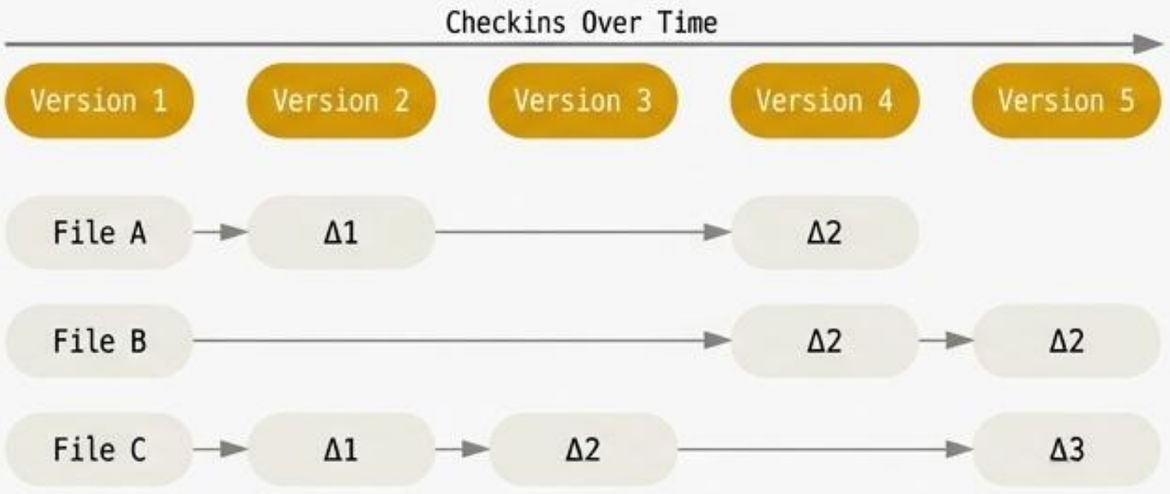
Domina Git y GitHub

Conceptos, Comandos y Flujo de Trabajo

NotebookLM

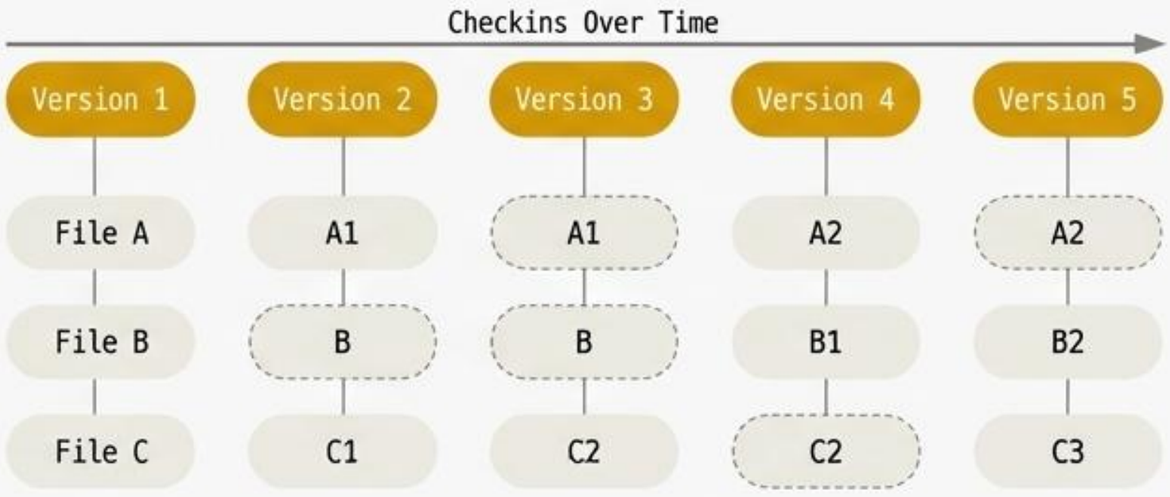
El paradigma de Git: Instantáneas, no diferencias

Otros VCS



Almacenan información como una lista de cambios (deltas) en los archivos a lo largo del tiempo.

Git

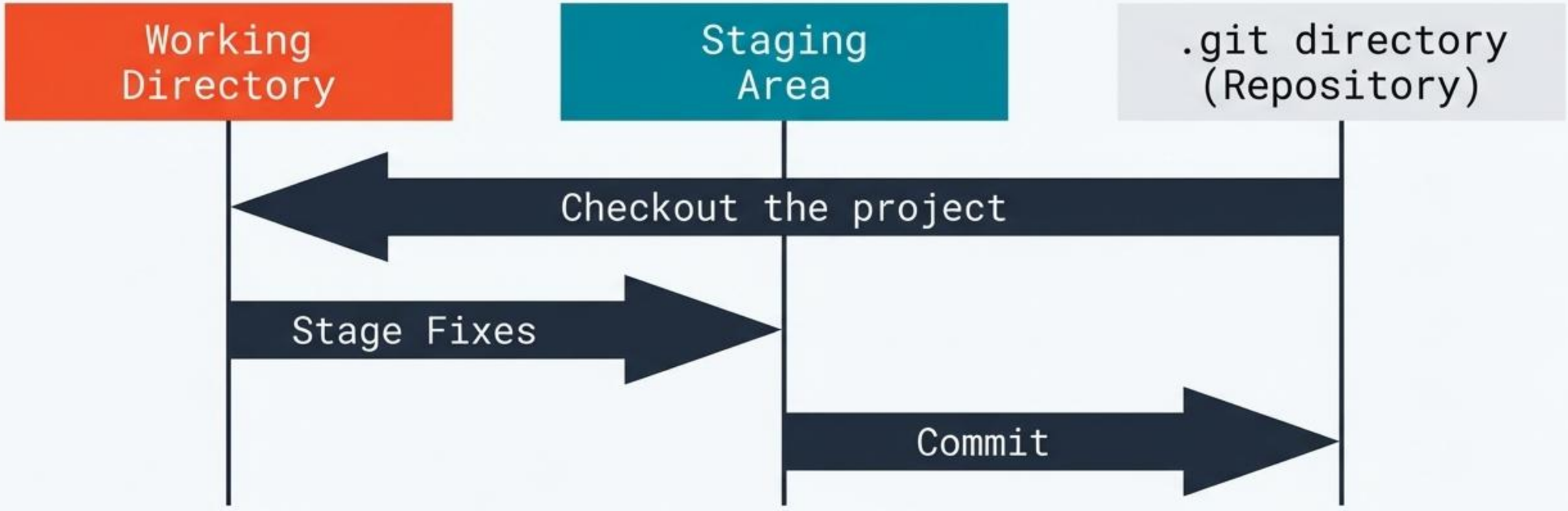


Guarda “instantáneas” (snapshots). Toma una foto del aspecto de todos tus archivos en ese momento y guarda una referencia.

Si un archivo no cambia, Git no lo almacena de nuevo; crea un enlace al archivo idéntico anterior.

NotebookLM

Los 3 estados fundamentales



Archivos extraídos de la base de datos listos para ser usados o modificados.

Estado: Modificado

Un archivo que almacena información sobre lo que irá en tu próxima confirmación.

Estado: Preparado (Staged)

Donde Git almacena los metadatos y la base de datos de objetos.

Estado: Confirmado (Committed)

NotebookLM

Primeros pasos: Configuración e Inicialización

Configurar Identidad

```
$ git config --global user.name "[tu_nombre]"
$ git config --global user.email "[tu_email]"
```

→ **Nota:** Se anexa de forma inmutable a tus transacciones de commit para la colaboración.

Crear Repositorios

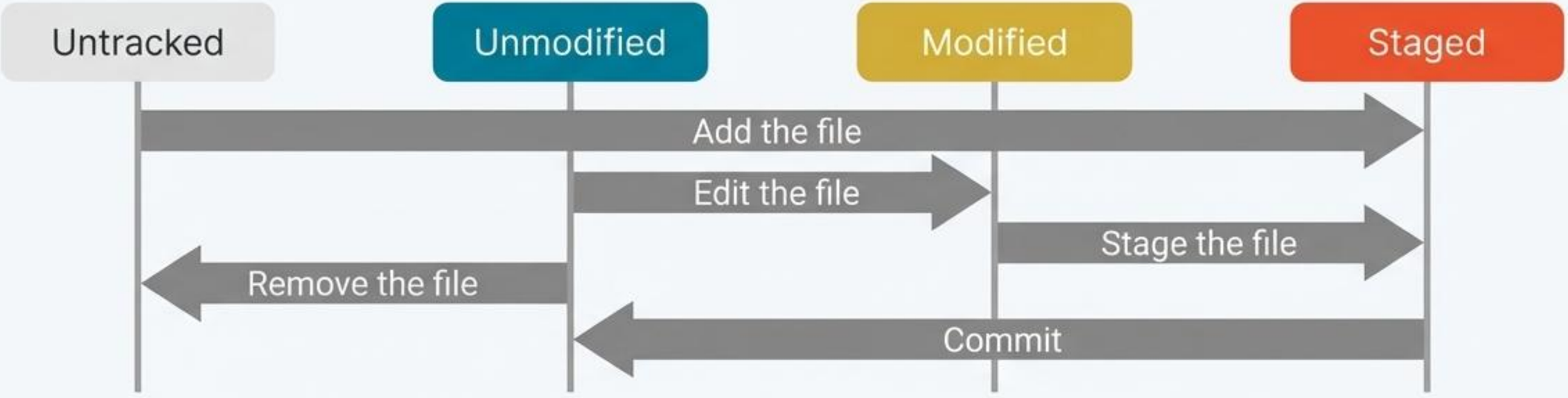
```
$ git init [nombre-proyecto]
$ git clone [url]
```

→ Crea un nuevo repositorio local vacío.

→ Descarga un proyecto y su historia completa desde un servidor.

NotebookLM

El ciclo de vida de los archivos



- **Untracked (Sin rastrear)**: Archivos nuevos no incluidos en el commit anterior.
- **Unmodified/Modified (Rastreados)**: Archivos que Git ya conoce.
- **Staged (Preparados)**: Listos para el próximo commit.

El Radar

```
$ git status     -> Muestra la lista de archivos nuevos, modificados o preparados.
Atajo: Usa $ git status -s para una vista compacta.
```

NotebookLM

Construyendo la instantánea: Add & Commit

Paso 1: Preparar (Add)

```
$ git add [archivo]
```

-> Toma una instantánea del archivo para preparar la versión.

Tip: Piensa en add como 'añadir este contenido a la próxima confirmación', no al proyecto en sí.

Paso 2: Confirmar (Commit)

```
$ git commit -m "[mensaje_descriptivo]"
```

-> Registra permanentemente la instantánea en el historial.

Atajo temporal:

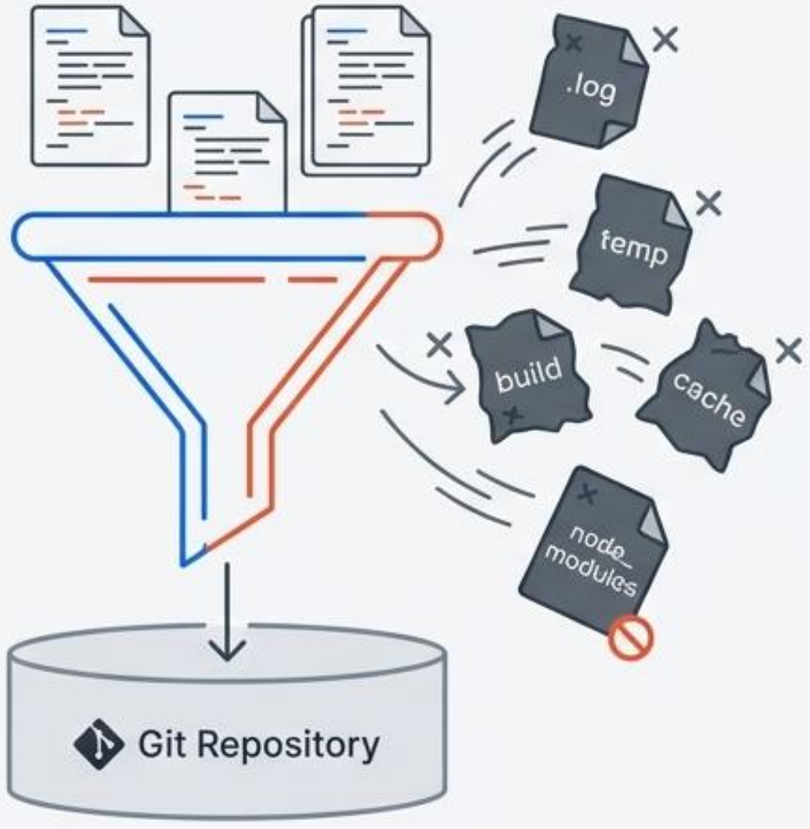
```
$ git commit -a -m "[mensaje]"
```

(Prepara todos los archivos rastreados modificados automáticamente, saltando git add).

NotebookLM

Mantener el repositorio limpio: .gitignore

Concepto: Un archivo de texto llamado .gitignore suprime la creación accidental de versiones de archivos temporales o autogenerados.




```
*.[oa] # Ignora archivos de objeto compilados
*~     # Ignora archivos temporales de editores
build/ # Ignora todo el directorio de compilación
*.log  # Ignora trazas de ejecución
```

NotebookLM

Inspeccionando el historial y los cambios

Viajar en el tiempo (git log)

```
$ git log --oneline --graph
```

-> Historial compacto con gráfico ASCII de ramas.

```
$ git log --stat
```

-> Muestra estadísticas de archivos modificados.

Ver los parches exactos (git diff)

```
$ git diff
```

-> Diferencias en el directorio de trabajo (aún no preparadas).

```
$ git diff --staged
```

-> Diferencias entre el área de espera y la última confirmación.

```
- merged in.
+ merged in. Also, split your changes.
```

NotebookLM

El poder de deshacer: Recuperación de errores

Modificar el último commit

```
$ git commit --amend
```

→ Agrega archivos olvidados o corrige el mensaje del commit anterior.

Quitar del Área de Preparación

```
$ git reset HEAD [archivo]
```

→ Saca el archivo de Staging, pero preserva tus modificaciones en disco.

Descartar cambios locales (⚠️ Peligro)

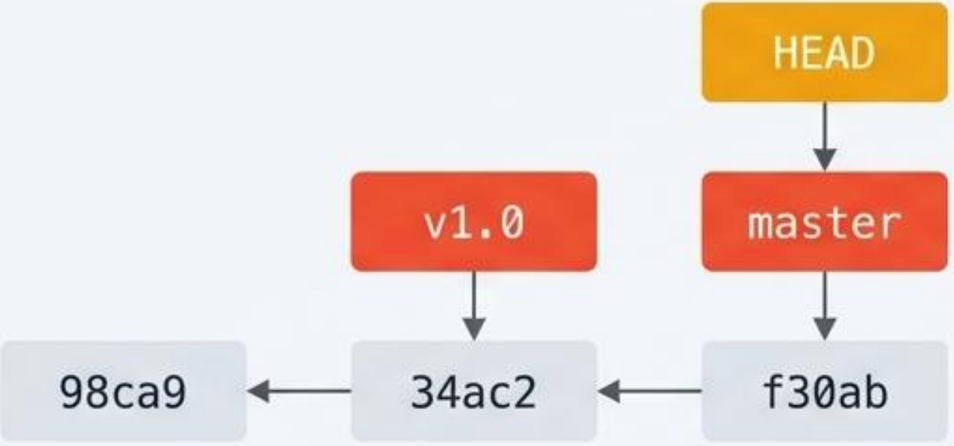
```
$ git checkout -- [archivo]
```

→ Sobrescribe el archivo con la última versión confirmada.

Los cambios no guardados se perderán.

NotebookLM

Desmitificando las Ramas (Branching)



- **Concepto Central:** En Git, crear una rama NO copia archivos. Una rama es simplemente un puntero móvil que apunta a un commit específico.
- **¿Qué es HEAD?:** Es un puntero especial que indica en qué rama local te encuentras actualmente.
- **La Ventaja Git:** Crear y destruir ramas es instantáneo. Git promueve crear ramas múltiples veces al día para experimentar.

NotebookLM

Navegando en paralelo

Listar Ramas

```
$ git branch
```

→ Enumera todas las ramas locales.

Crear y Saltar

```
$ git branch [nombre]
```

→ Crea una nueva rama (pero no salta a ella).

```
$ git checkout [nombre]
```

→ Mueve HEAD a la rama especificada y actualiza el directorio de trabajo.

Atajo Ninja: `$ git checkout -b [nombre]`

→ Crea y salta en un solo paso.

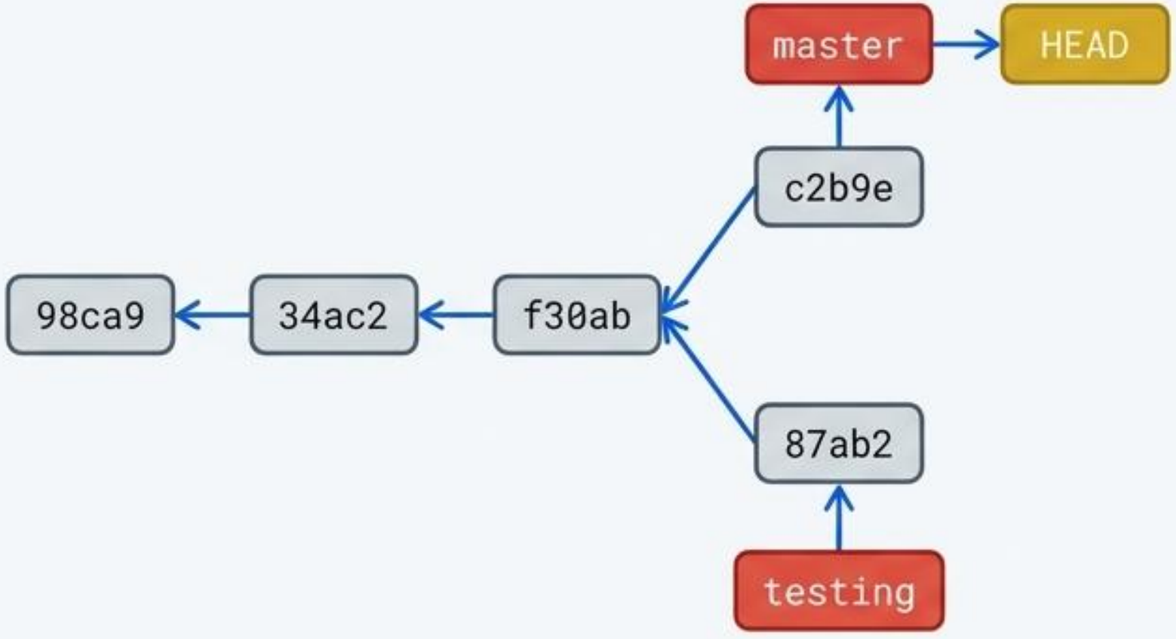
Limpieza

```
$ git branch -d [nombre]
```

→ Borra la rama una vez terminado el trabajo.

NotebookLM

Combinando esfuerzos: Merge




```
$ git merge [rama]
```

–> Combina el historial de la rama especificada con la rama actual (donde está HEAD).

Avance Rápido (Fast-forward):

Si la rama actual no ha divergido y está directamente "detrás" de la rama a fusionar, Git simplemente mueve el puntero hacia adelante.

Divergencia:

Si ambas ramas tienen nuevos commits, Git crea una nueva "confirmación de fusión" (*merge commit*) uniendo los historiales.

NotebookLM

El mundo distribuido: Conectando con Remotos

Concepto: Los remotos son versiones de tu proyecto hospedadas en Internet. El nombre por defecto del servidor principal suele ser origin.

Visualizar Conexiones

```
$ git remote -v
```

-> Muestra las URLs asociadas de lectura (fetch) y escritura (push).

Añadir un Remoto

```
$ git remote add origin [url]
```

-> Registra un nuevo marcador de repositorio remoto bajo el nombre 'origin'.

NotebookLM

Sincronización y colaboración

Traer cambios del exterior:

```
$ git fetch
```

→ Descarga todo el historial del repositorio remoto, pero no modifica tu trabajo local.

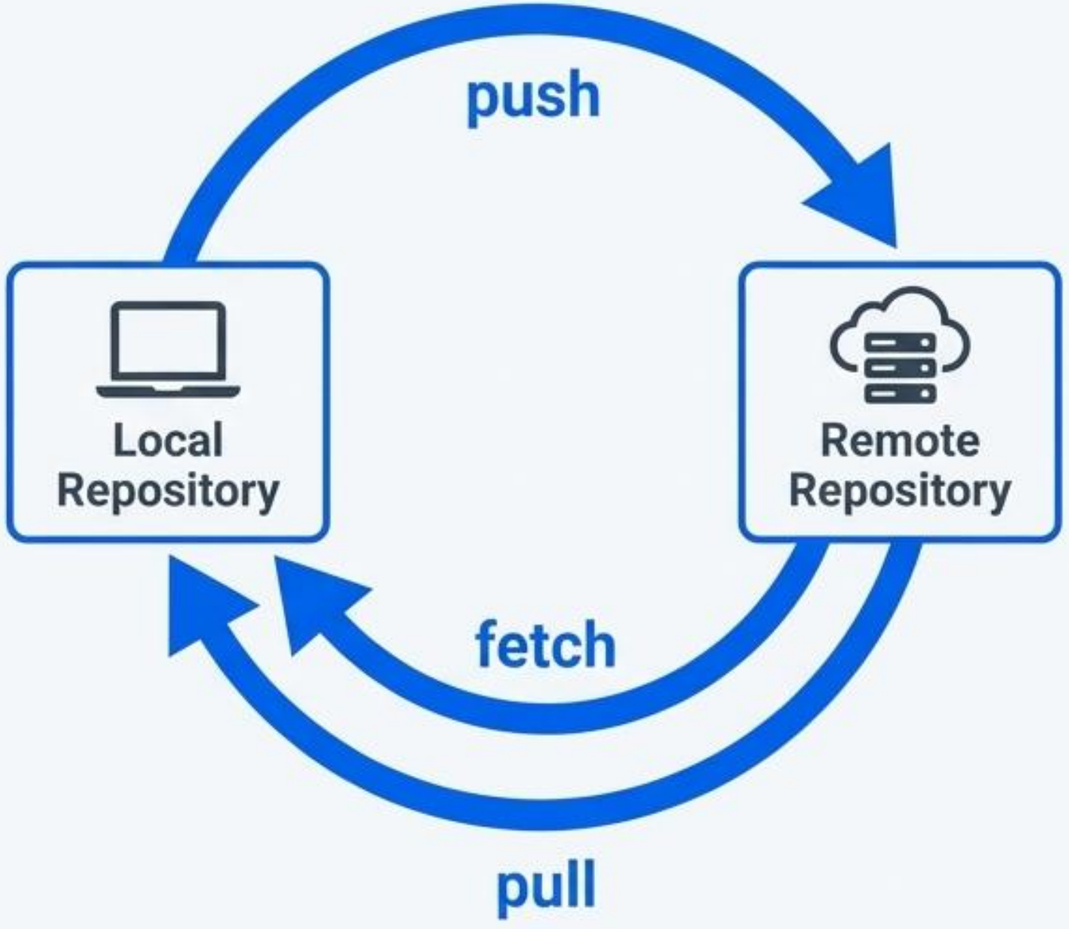
```
$ git pull
```

→ Descarga el historial e incorpora/combina automáticamente los cambios en tu rama actual.

Enviar tu trabajo:

```
$ git push origin [rama]
```

→ Carga todos los commits de tu rama local hacia el repositorio remoto en GitHub.



NotebookLM

Cheat Sheet: El Flujo de Trabajo Esencial

1. Iniciar & Configurar

```
git init
git clone
git config git config "emv"
```

2. Efectuar Cambios

```
git status
git add .
git commit -m "
git diff
```

3. Ramificar (Branching)

```
git branch
git checkout -b
git merge
```

4. Sincronizar (Remotos)

```
git fetch
git pull
git push
```

> Git guarda instantáneas seguras; no temas experimentar. Todo commit es recuperable.

NotebookLM